



A JAVA-BASED STUDY PLANNER

PROJECT REPORT

NAME – ANUSHKA CHAUDHARY

REGISTRATION NO. – 25BAI10817

SLOT – F11+F12

FACULTY NAME – PROF. KAMLESH CHANDRAVANSHI

1. INTRODUCTION

Students rarely deal with one subject at a time. In any given week there may be an assignment due in one course, a lab record pending in another, a unit test approaching in a third, and a project deadline somewhere behind all of them. Most students track this informally a note in a diary, a message in a class group, or simply memory. Work is then remembered late, smaller tasks are forgotten entirely, and priority is decided by whichever deadline feels nearest rather than by what actually matters most.

StudySync is a Java-based command line study planner built to address this. It allows a student to record an academic task along with its subject, deadline and priority, mark it as completed when the work is done, delete tasks that are no longer relevant, and view a summary of how much of the overall workload has been finished. The task list is stored in an ArrayList while the program runs and written to a local file using Java object serialization, so nothing is lost when the program is closed.

The project was developed to apply core Java concepts classes and objects, collections, control structures, exception handling and file input/output to a problem drawn from everyday academic life rather than to an isolated textbook exercise. The emphasis throughout was on simplicity, reliability and clean structure rather than on feature count.

1.1 Why This Idea?

The idea came from my own experience of losing track of deadlines across subjects. I looked at the alternatives before starting. General to-do applications have no concept of a subject, so academic work loses its context. Institutional learning platforms require accounts, internet access and setup effort that outweighs the benefit for a single student keeping a personal list. What was missing was something small, immediate and offline.

As a Java learner I also wanted to build something that was:

- Genuinely useful in day-to-day student life
- Simple enough to finish and explain completely
- Directly aligned with the concepts covered in the course
- Free of any external library, so it runs anywhere a JDK is installed

1.2 Scope of the Project

The scope of this project includes:

- Adding tasks with subject, task name, deadline and priority
- Displaying all stored tasks with their completion status
- Marking tasks as completed
- Deleting tasks that are no longer needed
- Reporting study progress as counts and a percentage
- Saving and reloading data automatically using serialization

The system is, however, limited to:

- Command-line interaction only, with no graphical interface
- Text-based deadlines, with no date calculation or overdue detection
- A single user on a single machine, with no login or networking

It therefore serves as a working prototype that demonstrates how a personal academic task manager can be designed and implemented.

2. PROBLEM STATEMENT

This project began from a real and repeated observation in my own academic routine.

How can a student's academic workload be captured, prioritised and tracked through a simple offline application, so that tasks, deadlines and completion progress are visible in one place without depending on internet access or external software?

When several subjects compete for attention, the difficulty is not doing the work but knowing what is outstanding. A structured list that already holds the subject, the deadline and the priority of each task and that can report progress on demand, removes that uncertainty. I also wanted completed tasks to remain visible rather than disappear, so that progress feels measurable instead of the list simply shrinking.

This problem maps directly onto the concepts studied in the course:

- Classes and objects- to model a task as a single entity with four attributes and a status
- ArrayList- to store a collection of tasks whose size is not known in advance
- Methods- to separate each feature of the program cleanly
- Loops- to drive the menu and to iterate over the task list
- Conditionals- to route menu choices and validate task numbers
- Exception handling- to keep the program running when input or file access fails
- File handling and serialization- to make the data persist between runs

3. APPROACH TO SOLVE THE PROBLEM

The problem was approached by designing a menu driven command-line application built up in the following stages:

1. Data Modelling

- Define a Task class holding subject, name, deadline, priority and a boolean done flag
- Make the class implement Serializable so whole objects can be written to disk

2. Task Management

- Maintain an ArrayList<Task> as the single source of truth while the program runs
- Provide separate methods to add, display, complete and delete tasks

3. Progress Reporting

- Iterate the list to count completed tasks
- Calculate a percentage using double division, guarding against an empty list

4. Data Persistence

- Save the entire list to tasks.dat after every change rather than only at exit

- Load the file at startup, falling back to an empty list when no file exists

5. User Interaction

- Use a do-while loop so the menu repeats until Exit is chosen
- Display the task list before asking which task to complete or delete
- Validate every numeric input and confirm every action with a short message

This approach keeps the program simple to follow, functional in each of its features and straightforward to extend later.

4. KEY DECISIONS MADE

Several design decisions shaped the final program:

- Command-line interface instead of a GUI- Keeps attention on program logic and data handling rather than on interface code
- A dedicated Task class instead of parallel arrays- Keeps the five attributes of one task together as a single object, which is far easier to store, pass around and extend
- ArrayList instead of a fixed array- The number of tasks is unknown in advance, and ArrayList handles growth and removal automatically
- Serialization instead of writing a text format- The whole list, including the boolean status of each task, is saved and restored in a single call with no parsing code to write
- Saving after every change, not only on exit- Protects the data if the program is closed unexpectedly
- Completion as a status flag rather than removal- Finished work stays visible so progress can be measured, with Delete kept as a separate deliberate action
- Reading input with `nextLine()` throughout- Avoids the leftover-newline problem caused by mixing `nextInt()` and `nextLine()`, and allows multi-word task names
- One method per menu option- Improves readability and makes each feature testable on its own
- Together these decisions balanced simplicity against functionality and kept the program well within the scope of the course.

5. CHALLENGES FACED

A number of difficulties came up during development:

- Handling a missing data file on the very first run, where loading raised an exception instead of simply starting empty. This was solved by catching the exception and initialising a fresh ArrayList.
- Making Task serializable- The initial attempt to save the list failed until the class was made to implement the Serializable interface.
- A `NumberFormatException` whenever text was typed at a numeric prompt, which was fixed by wrapping `Integer.parseInt()` in try-catch.
- Task numbers out of range causing `IndexOutOfBoundsException`, resolved by range-checking the number before using it as an index.

- Integer division returning 0% for partial progress, corrected by casting to double before dividing.
- Deciding when to save, and settling on saving after every modifying operation so data is never left only in memory.
- Handling the empty-list case in Complete, Delete and Progress so that none of them crashed or reported nonsense.
- The unchecked cast warning on readObject(), which had to be understood rather than ignored.

Working through these problems improved my debugging habits considerably and taught me to think about what a user might type rather than only about what the program expects.

6. WHAT I LEARNED

6.1 Technical Skills

Through this project I learned:

- Designing a class to model real-world data and creating objects from it
- Working with ArrayList- adding, retrieving, removing and iterating
- Implementing file handling with object streams
- Using Java serialization to persist an entire collection
- Using loops and conditionals to build a menu-driven program
- Exception handling with try-catch to prevent crashes
- Using classes and interfaces such as:
 - `java.util.ArrayList` and `java.util.Scanner`
 - `java.io.Serializable`, `ObjectOutputStream` and `ObjectInputStream`

6.2 Problem Solving and Design

- Breaking a single problem into independent features
- Designing the data model before writing any logic
- Thinking about the sequence a user actually follows, not just the code path
- Choosing the right data structure for the requirement
- Anticipating invalid input instead of assuming correct input

6.3 Software Development Process

- Writing and testing the program feature by feature rather than all at once
- Reading and interpreting stack traces to locate the real cause of a failure
- Testing edge cases such as an empty list, a wrong number and a missing file
- Refactoring repeated code into reusable methods
- Documenting limitations honestly rather than hiding them

This project gave me a much clearer picture of how a small piece of real software is actually built, debugged and finished.

7. JAVA CONCEPTS APPLIED

Java Concept	Where it appears in my code
Class and Object	The Task class models one study task; objects are created in addTask()
ArrayList<Task>	taskList — the dynamic in-memory store for all tasks
Serializable interface	Task implements Serializable so objects can be written to a file
Static methods	Each menu feature is a separate static method (addTask, showTasks, etc.)
Constructor	Task(subject, name, deadline, priority) initialises a task as not done
switch statement	Routes the user's menu choice to the correct method
do-while loop	Keeps the main menu running until the user selects Exit
for loop	Indexed loop in showTasks() to display numbered tasks
for-each loop	Counts completed tasks in showProgress()
if / else	Range checks on task numbers and empty-list guards
Ternary operator	t.done ? "Completed" : "Pending" for status display
try / catch	Catches invalid numeric input and all file I/O failures
Integer.parseInt()	Converts the typed menu choice and task number into an int
Scanner (nextLine)	Reads all user input as complete lines
Type casting	(double) casting for accurate percentage calculation; (ArrayList<Task>) on load
File I/O streams	FileOutputStream / ObjectOutputStream to save, and the input pair to load
Object serialization	writeObject() and readObject() persist the whole list in tasks.dat
String concatenation	Building all console output messages

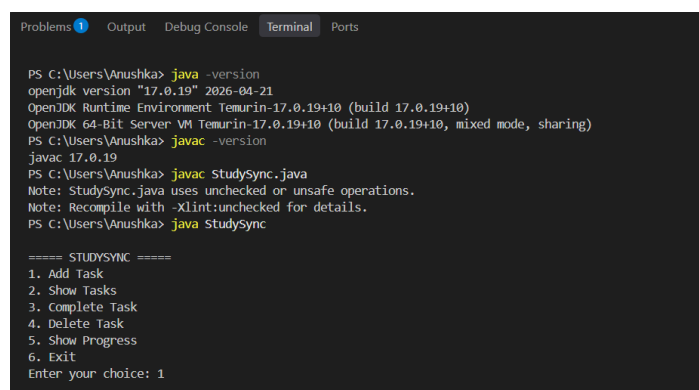
8. HOW TO RUN THE PROJECT

- Install the Java Development Kit (JDK 8 or above) and verify it with `java -version`
- Download or clone the project folder containing StudySync.java
- Open a terminal or command prompt inside that folder
- Compile the program with the command: `javac StudySync.java`
- Run the program with the command: `java StudySync`
- Choose an option from the menu by entering a number from 1 to 6
- Always exit using option 6 so that data is saved correctly

A file named tasks.dat is created automatically in the same folder once the first task is added. This file holds all saved tasks and must be kept alongside the program.

9. OUTPUTS

The screenshots below show the program running through each of its features.



```
Problems Output Debug Console Terminal Ports
PS C:\Users\Anushka> java -version
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment Temurin-17.0.19+10 (build 17.0.19+10)
OpenJDK 64-Bit Server VM Temurin-17.0.19+10 (build 17.0.19+10, mixed mode, sharing)
PS C:\Users\Anushka> javac -version
javac 17.0.19
PS C:\Users\Anushka> javac StudySync.java
Note: StudySync.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
PS C:\Users\Anushka> java StudySync

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 1
```

```
Problems 1 Output Debug Console Terminal Ports

--- Add Task ---
Subject: Java
Task name: OOP Assignment
Deadline: 18-09-26
Priority (High/Medium/Low): High
Task added successfully.

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 2

--- Your Tasks ---

Task 1
Subject: Java
```

```
Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: Pending

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 3

--- Your Tasks ---

Task 1
Subject: Java
Name: OOP Assignment
```

```
Problems 1 Output Debug Console Terminal Ports

--- Your Tasks ---

Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: Pending

Enter task number to complete: 1
Task completed!

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 2
```

```
Problems 1 Output Debug Console Terminal Ports

Enter your choice: 2

--- Your Tasks ---

Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: completed

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 5

--- Study Progress ---
```

```
Problems 1 Output Debug Console Terminal Ports
5. Show Progress
6. Exit
Enter your choice: 5

--- Study Progress ---
Total tasks: 1
Completed: 1
Pending: 0
Progress: 100.0%

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 1

--- Add Task ---
Subject: Operating System
```

```
Problems 1 Output Debug Console Terminal Ports

--- Add Task ---
Subject: Operating System
Task name: Complete Module 3
Deadline: 21-09-26
Priority (High/Medium/Low): High
Task added successfully.

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 5

--- Study Progress ---
Total tasks: 2
Completed: 1
Pending: 1
```

```
Problems 1 Output Debug Console Terminal Ports

Completed: 1
Pending: 1
Progress: 50.0%

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 3

--- Your Tasks ---

Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: Completed
```

```
Problems 1 Output Debug Console Terminal Ports

Task 2
Subject: Operating System
Name: Complete Module 3
Deadline: 21-09-26
Priority: High
Status: Pending

Enter task number to complete: 2
Task completed!

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 5

--- Study Progress ---
Total tasks: 2
```

```

Problems 1 Output Debug Console Terminal Ports

--- Study Progress ---
Total tasks: 2
Completed: 2
Pending: 0
Progress: 100.0%

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 1

--- Add Task ---
Subject: DLCA
Task name: Practical Task 2
Deadline: 24-09-26
Priority (High/Medium/Low): High
Task added successfully.

```

```

Problems 1 Output Debug Console Terminal Ports

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 3

--- Your Tasks ---

Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: Completed

Task 2
Subject: Operating System

```

```

Problems 1 Output Debug Console Terminal Ports

Priority: High
Status: Completed

Task 2
Subject: Operating System
Name: Complete Module 3
Deadline: 21-09-26
Priority: High
Status: Completed

Task 3
Subject: DLCA
Name: Practical Task 2
Deadline: 24-09-26
Priority: High
Status: Pending

Enter task number to complete: 2
Task completed!

===== STUDYSYNC =====

```

```

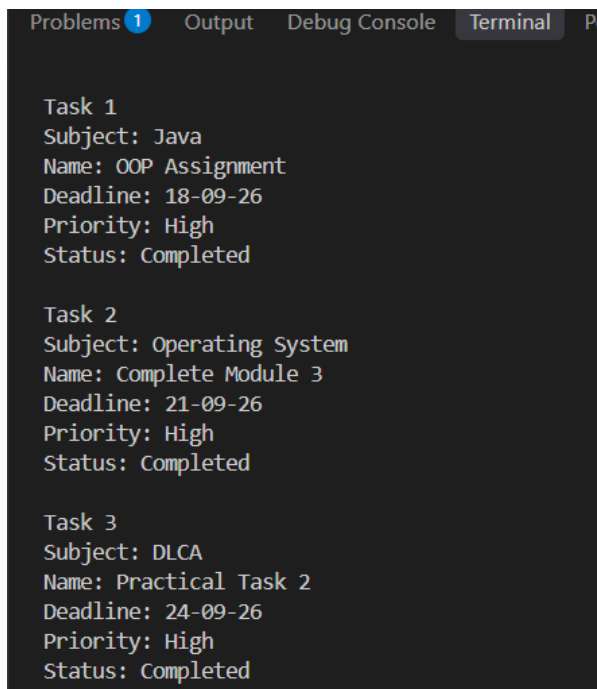
Enter task number to complete: 2
Task completed!

===== STUDYSYNC =====
1. Add Task
2. Show Tasks
3. Complete Task
4. Delete Task
5. Show Progress
6. Exit
Enter your choice: 5

--- Study Progress ---
Total tasks: 3
Completed: 2
Pending: 1
Progress: 66.66666666666666%

===== STUDYSYNC =====
1. Add Task
2. Show Tasks

```

A screenshot of a terminal window with a dark background. The terminal has tabs at the top: 'Problems' (with a blue circle containing the number 1), 'Output', 'Debug Console', and 'Terminal'. The 'Terminal' tab is active. The text in the terminal is as follows:

```
Task 1
Subject: Java
Name: OOP Assignment
Deadline: 18-09-26
Priority: High
Status: Completed

Task 2
Subject: Operating System
Name: Complete Module 3
Deadline: 21-09-26
Priority: High
Status: Completed

Task 3
Subject: DLCA
Name: Practical Task 2
Deadline: 24-09-26
Priority: High
Status: Completed
```

10. LIMITATIONS

- The interface is command-line only, which may not be user-friendly for non-technical users.
- Deadlines are stored as plain text, so the program cannot sort by date or detect overdue tasks.
- An invalid date such as '32 February' would be accepted without complaint.
- Priority is neither validated nor used- tasks are never ordered by importance.
- There is no option to edit a task; a mistake must be deleted and re-entered.
- The list cannot be searched, filtered or sorted and is always shown in insertion order.
- No reminders or notifications are generated as deadlines approach.
- Data is local to one machine and one user with no authentication or encryption.
- tasks.dat is a binary file, so it cannot be read, printed or shared as a list.
- Serialization is tied to the Task class, so changing that class can make older saved files unreadable.
- Exceptions are caught broadly as Exception, so the precise cause of a failure is not always reported.

11. FUTURE IMPROVEMENTS

The project can be extended in several directions to make it more capable and closer to a real application:

- Replacing the text deadline with LocalDate would allow date validation, sorting by due date and automatic flagging of overdue tasks.

- Sorting and filtering by priority, subject or deadline would make a long list far more usable.
- An Edit Task option would let a student correct an entry instead of deleting and retyping it.
- Restricting priority input to High, Medium or Low with re-prompting would keep the data consistent.
- Storing data as CSV or JSON would make the file readable and shareable, and would survive changes to the Task class.
- A per-subject progress breakdown would show where attention is most needed.
- A graphical interface built with JavaFX or Swing and later a mobile or web version, would make the planner accessible to any student.
- Reminders or desktop notifications ahead of deadlines would turn the planner from a passive record into an active assistant.
- User accounts with authentication would allow several students to share one installation safely.
- Study-time tracking per task could produce weekly reports on where time is actually spent.
- Moving from file storage to a database such as SQLite or MySQL through JDBC would improve scalability.
- Adding JUnit tests would let the add, complete, delete and progress logic be verified automatically after every change.

Taken together these improvements would turn the prototype into a complete and genuinely practical study management tool.

12. CONCLUSION

This project helped me understand how the concepts learned in Java can be brought together to solve a real problem rather than practised in isolation. Instead of writing separate exercises on classes, collections, loops and file handling, I had to combine all of them into one program in which every part depended on the others working correctly.

One of the main lessons was that writing a program is not only about making it produce the right output. It is equally about structure and clarity- giving each feature its own method, choosing a data structure that suits the requirement and naming things so that the code can be read later without effort.

I also learned to look at the program from the user's side. Most of my debugging time went not into the features themselves but into what happens when something unexpected occurs: text typed where a number was expected, a task number that does not exist, an empty list, or a missing data file. Handling those cases with clear messages instead of a stack trace is what separates a program that works from a program that can be used.

Although StudySync is a simple command-line application, building it from an idea through to a working, persistent program gave me a clear sense of how software is actually developed and increased my confidence in taking on larger projects. It strengthened my Java fundamentals, improved the way I approach a problem before coding it and left me interested in extending the project further with dates, a graphical interface and database storage.

13. REFERENCES

- Java code compiled and executed using the Java Development Kit (JDK) from the terminal
- Oracle Java official documentation — `java.util.ArrayList`, `java.util.Scanner`, `java.io.Serializable`, `ObjectOutputStream` and `ObjectInputStream`
- Oracle Java Tutorials — Collections, Exception Handling and Object Serialization
- Class notes, lab exercises and course materials provided during the module
- Project source file: `StudySync.java`